

Killing Flaky Tests & Running AI Agents for Hours

How an AI Agentic QA framework — powered by **Claude Opus 4.8** and a disciplined agent **harness** — eliminates test flakiness, sustains long-running autonomous runs, and keeps results accurate.



TEST ENGINEER

Jayesh Pagar



MANAGER

Chaitanya Tipre



PROJECT MANAGER

Sudharshan Lokhande

240

Automated Tests

0

Flaky-retry Reliance*

3

Autonomous Agents

1M

Context Window

**Suite validated to pass at retries = 0; one retry is kept only as a safety net for transient staging blips.*

Two failure modes that break automation



Flaky Tests

Tests that pass and fail without a code change destroy trust. In a real-world case the root cause was authentication: **a token-based auth provider stored sessions in localStorage and rotated refresh tokens on every refresh**, so reusing a saved storageState file went stale and failed intermittently — plus concurrent logins hit rate limits.



Long-Running Agents

An AI agent generating and healing hundreds of tests can run for a long time. Without help it loses earlier context, repeats work, drifts off task, or dies mid-run — wasting tokens and producing inconsistent output.

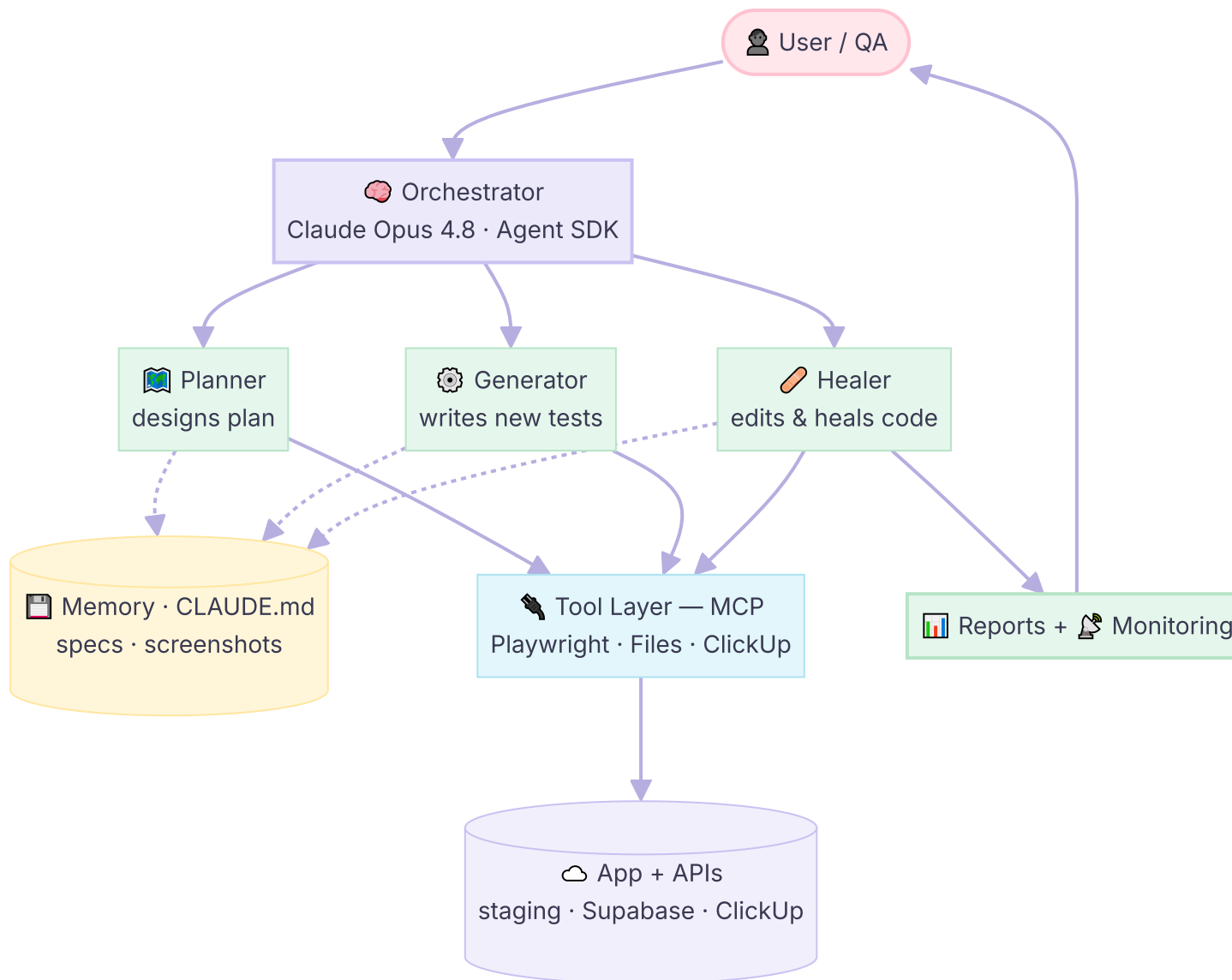


The goal

Make every run **deterministic** (no flakiness), **durable** (survives long autonomous sessions), and **accurate** (results you can publish to stakeholders).

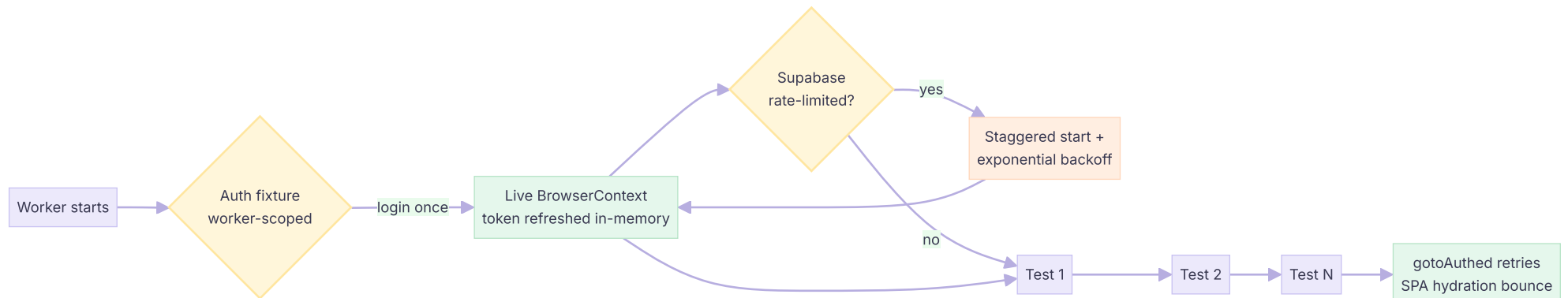
One orchestrator, three specialist agents

The Claude Opus 4.8 orchestrator decomposes a user story and delegates to least-privilege sub-agents that collaborate through a shared file "blackboard".



Planner & Generator hold no code-editing tools (can't corrupt existing code); only the Healer edits files. Only the ClickUp CLI talks to ClickUp.

How flaky tests are eliminated



Worker-scoped auth

Each worker logs in **once** into a live context kept alive for its lifetime — tokens refresh in memory, never from a stale file.



Backoff & stagger

Staggered worker startup + exponential backoff absorbs Supabase concurrent-login rate limits.



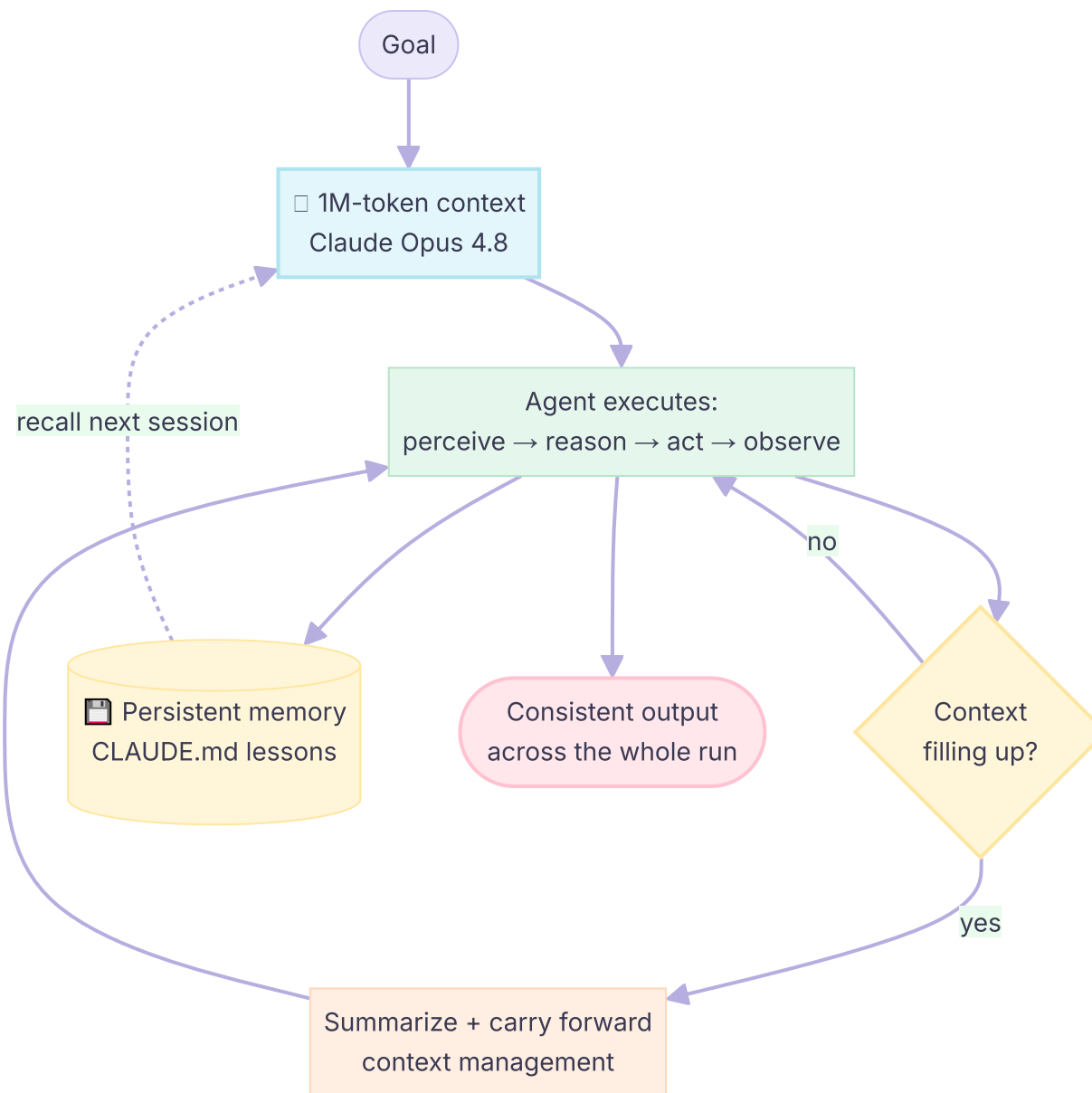
No blind waits

`gotoAuthed()` retries the SPA hydration bounce; **zero** `waitForTimeout()` — waits are tied to real signals.

*Result: the suite passes at **retries = 0**; the single configured retry is a safety net, not a flake mask.*

How long-running agents stay on track

The model and the harness together let agents work for hours without losing the thread.



1M context

Opus 4.8's large window holds the whole task — plan, code and failures — in view at

Context management

When context grows, the harness summarizes and carries state forward so

Persistent memory

Hard-won lessons (e.g. the auth-flake fix) are written to memory and recalled in future

once.

work continues seamlessly.

runs.

*Roadmap: a dedicated **Long-Running Agent Harness** adds heartbeats, checkpoints & auto-recovery for multi-hour, unattended runs.*

Why the results are trustworthy

✗ Without the framework

- Random pass/fail, retries hide bugs
- Selectors patched inside specs → drift
- Agents forget context, repeat work
- Numbers nobody trusts

✓ With the framework

- Deterministic auth, real-signal waits
- Fixes layered: selectors live in Page Objects
- Self-healing repairs the right layer
- Evidence-backed, auditable results

5

Modules

240

Test Cases

19

Page Objects

102

Evidence Shots

Every defect is traced requirement → test → pass/fail → ClickUp ticket with screenshots.

TAKEAWAY

Agentic QA is working today — not someday

By pairing **Claude Opus 4.8** with a disciplined agent **harness**, flakiness is engineered out, long autonomous runs stay coherent, and the quality signal is accurate enough to put in front of a CTO.



Reliable

Zero-flake auth, no blind waits.



Durable

1M context + memory + self-healing.



Accurate

Traceable, evidence-backed results.

► A reference pattern you can apply to any web app and any test suite.



TEST ENGINEER

Jayesh Pagar



MANAGER

Chaitanya Tipre



PROJECT MANAGER

Sudharshan Lokhande

